

AI 7993 –AI Capstone | Section W01/01 | Spring 2026
Final Project Report

Project ID: AC-1

NN2Logic
Neural Network to Logic Compiler

Deepthi Kondreddy
Pragya Mishra
Sukumar Muthusamy

May 4, 2026

GitHub: <https://github.com/sukumarmuthusamy/nn2logic>
Youtube link: <https://youtu.be/cKJKUGYuTXc>

Table of Contents

Abstract

1. Introduction

2. Requirements

2.1 Functional Requirements

2.2 Non-Functional Requirements

2.3 System Constraints

3. Analysis

3.1 Problem Analysis

3.2 Dataset Analysis

4. Design

4.1 System Architecture

4.2 Data Structure Design

4.3 Algorithm Design

OBDD Pseudo-Polynomial Compilation

NNF Composition

SDD Compilation

4.4 Network Configuration Design

5. Development

5.1 Phase 1 — Prototype (Single Neuron, v0.1)

5.2 Phase 2 — Multi-Layer XOR Network (v0.2)

5.3 Phase 3 — SDD Integration (v0.3–v0.5)

5.4 Phase 4 — Scaled Pipeline (v1.0)

5.5 Version History

6. Test Plan and Report

6.1 Test Plan

6.2 Test Report

6.3 Pipeline Stage Verification Results

7. Version Control

7.1 Key Repository Files

8. Conclusion

8.1 Future Work

Appendix A — Technologies and Tools

Appendix B — Process Flow Description

B.1 Four-Phase Process Flow

B.2 Architecture Description

Appendix C — Project Plan

Appendix D — Screen Mock-ups and Outputs

Appendix E — System Configurations

Appendix F — Algorithms and Pipeline Logic

Appendix G — Test plan and Results

Appendix H — Dataset Truth Tables

Abstract

NN2Logic is a pipeline that converts trained Binary Neural Networks (BNNs) into formal Boolean logic representations, enabling symbolic verification and interpretability. The system transforms neural models through a sequence of representations—Arithmetic Circuits (AC), Negation Normal Form (NNF), and Sentential Decision Diagrams (SDD)—allowing efficient logical queries such as model counting, robustness analysis, and prime implicant extraction.

The pipeline was evaluated on Boolean datasets including XOR, AND, OR, and Majority3. Results demonstrate complete traceability between the neural network and its symbolic equivalent, with exact logical consistency preserved at every stage. The project shows that neural network behavior can be translated into verifiable logic without approximation, making formal verification feasible for binary networks.

1. Introduction

Neural networks have demonstrated remarkable performance across a wide range of applications. However, despite their accuracy, they are often criticized for their lack of interpretability. Their internal representations consist of numerical weights and activations, making it difficult to understand or verify how decisions are made. This limitation becomes particularly critical in domains where reliability and explainability are essential.

This project, **NN2Logic**, addresses this challenge by introducing a structured pipeline that converts a trained Binary Neural Network (BNN) into a formal Boolean logic representation. The primary goal is to bridge the gap between statistical learning models and symbolic reasoning systems, enabling formal verification and interpretability.

The pipeline transforms a trained neural network through multiple intermediate representations, including Arithmetic Circuits (AC), Negation Normal Form (NNF), and ultimately Sentential Decision Diagrams (SDD). These representations allow efficient querying of the network's behavior, including model counting, robustness analysis, and logical explanation extraction.

The system is evaluated using simple Boolean datasets such as XOR, AND, OR, and a three-input Majority function. These datasets provide a controlled environment for validating correctness while demonstrating the pipeline's ability to handle both linear and non-linear decision boundaries.

2. Requirements

2.1 Functional Requirements

The system is designed to perform several key functions. It must be able to train a Binary Neural Network using TensorFlow/Keras and extract its learned parameters, including weights and biases. These parameters are then used to construct Arithmetic Circuits that exactly represent each neuron's computation.

The system must compile these Arithmetic Circuits into Boolean representations using Negation Normal Form and combine them into a single end-to-end logical circuit. This combined representation is further compiled into a Sentential Decision Diagram to support efficient logical queries.

Additionally, the system must verify correctness at each stage by comparing outputs across all possible input combinations. It should also support interpretability features such as robustness analysis, model counting, and extraction of prime implicants.

2.2 Non-Functional Requirements

Beyond functionality, the system must ensure complete traceability between the neural network and its symbolic representation. It must handle floating-point precision issues reliably and support multiple datasets and network configurations without requiring code changes.

The system should also be robust to training variability by incorporating retry mechanisms and should handle external library limitations gracefully without crashing.

2.3 System Constraints:

The system operates under certain practical constraints that affect its scalability and deployment. A major limitation is the exponential growth in input combinations, which makes exhaustive evaluation feasible only for relatively small input sizes, typically up to around 20 input variables (i.e., 2^{20} combinations). This restricts the system from being applied directly to large-scale neural networks without approximation techniques.

In addition, the compilation of Sentential Decision Diagrams (SDDs) depends on platform-specific support, as the PySDD library is primarily compatible with Linux and macOS environments. Users on Windows systems may need to rely on alternatives such as WSL or cloud-based environments like Google Colab.

Another constraint arises from the inherent complexity of OBDD representations, where the number of paths can grow exponentially with network depth and width, making very large hidden layers computationally expensive to analyse exactly. Despite these limitations, the system remains effective for small to medium-scale networks where exact symbolic verification is required.

3. Analysis

3.1 Problem Analysis

The fundamental challenge is bridging the gap between neural networks, which operate on continuous real-valued weights and differentiable activations, and formal verification tools, which require discrete Boolean representations. A naive discretisation that rounds weights without tracking error would destroy the network's decision boundary. The pipeline must therefore translate the network's exact arithmetic into Boolean logic without any approximation.

For XOR, the network must learn a non-linear decision boundary that no single linear classifier can represent making it the canonical test case for multi-layer logic extraction. For 3-input Majority, two hidden layers are required, making it the most structurally demanding benchmark in the test suite.

3.2 Dataset Analysis

The system is evaluated on a set of standard logical benchmark datasets, including XOR, AND, OR, and Majority-3, to assess its ability to learn both linearly separable and non-linearly separable functions. Each dataset consists of binary input combinations, with sample sizes determined by the number of input variables (2 inputs resulting in 4 samples, and 3 inputs resulting in 8 samples). The distribution of positive outputs varies across datasets, ranging from balanced cases such as XOR and Majority-3 (50% positive outputs) to more skewed distributions such as AND (25%) and OR (75%).

From a separability perspective, AND and OR are linearly separable functions and can, in principle, be modelled using simple decision boundaries, whereas XOR and Majority-3 are not linearly separable and require deeper or more expressive representations. This mix of datasets provides a comprehensive evaluation setting, allowing the system to be tested on both simple logical mappings and more complex non-linear decision structures. Overall, this dataset selection ensures that the pipeline is assessed across varying levels of logical complexity and representational difficulty.

Dataset	Inputs	Samples	Positive Outputs	Linearly Separable
XOR	2	4	2 / 4 (50%)	No ✗
AND	2	4	1 / 4 (25%)	Yes ✓
OR	2	4	3 / 4 (75%)	Yes ✓
Majority3	3	8	4 / 8 (50%)	No ✗

Table 1: Dataset Analysis Summary

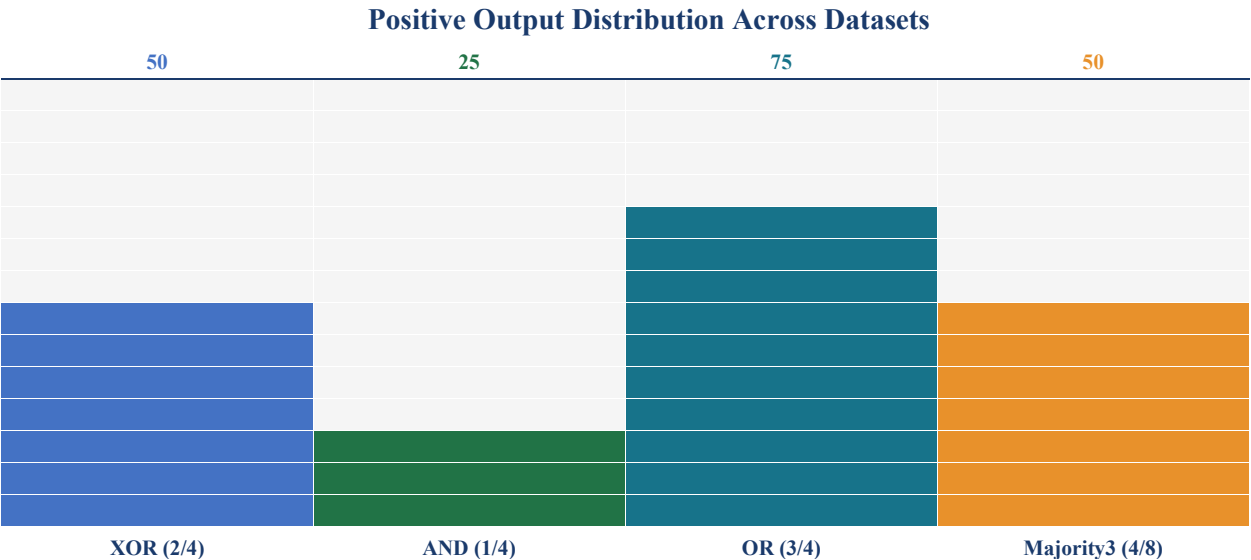


Figure 1: Positive Output Distribution Across Datasets

This figure visualizes the class distribution across the four benchmark datasets used in the evaluation. It highlights the proportion of positive outputs for each dataset—XOR (50%), AND (25%), OR (75%), and Majority-3 (50%). This comparison helps illustrate the varying levels of class imbalance and logical complexity present in the datasets.

XOR and Majority-3 serve as the most significant test cases, as they require hidden layers to capture non-linear decision boundaries, thereby validating the full multi-layer pipeline. In contrast, AND and OR are linearly separable and are used as simpler baseline cases for verifying the correctness of the overall framework.

4. Design

4.1 System Architecture

The pipeline is structured as a seven-stage sequence. Each stage transforms the network representation into a progressively more structured and queryable form:

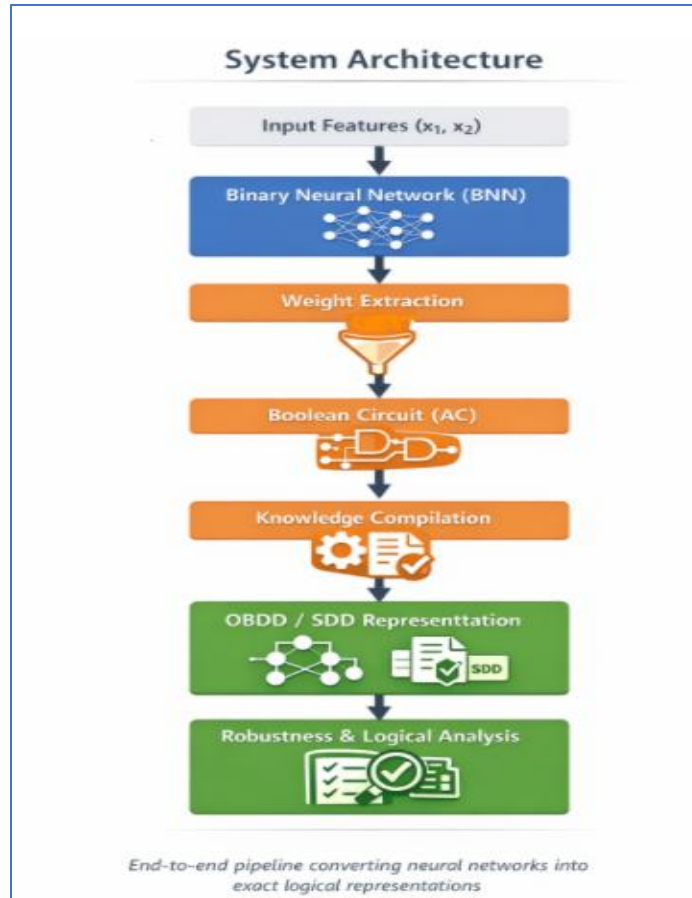


Figure 2: NN2Logic Seven-Stage Pipeline Architecture

4.2 Data Structure Design

The NN2Logic pipeline relies on a set of core data structures to represent and manipulate intermediate computations throughout the transformation process. At the arithmetic level, each neuron is modelled using an **ACNode**, which represents a node within an Arithmetic Circuit. These nodes capture fundamental operations such as inputs, constants, multiplication, addition, and the step activation function, allowing the exact numerical behaviour of the neural network to be preserved. At the symbolic level, the system uses **NNFNode** objects to represent elements of a Boolean circuit. These nodes operate using logical constructs such as literals, conjunctions (AND), and disjunctions (OR), with signed indices used to indicate variable polarity. To organize these nodes, the system defines two container structures: **AC**, which manages collections of ACNodes along with their connections and output node, and **NNF**, which similarly maintains NNFNodes while tracking the number of variables involved. Together, these data structures provide a structured and consistent way to transition from arithmetic representations to Boolean logic, ensuring both correctness and traceability across the pipeline.

4.3 Algorithm Design

OBDD Pseudo-Polynomial Compilation

The OBDD algorithm processes each weight variable sequentially, maintaining a dictionary of accumulated integer sums mapped to the list of literal paths that produced them. At each variable, the algorithm branches:

the variable-false path leaves the accumulated sum unchanged; the variable-true path increases it by the scaled integer weight. After all variables are processed, paths whose accumulated sum satisfies the threshold condition (≥ 0 after incorporating the bias) are collected as AND terms under a top-level OR, yielding a complete DNF formula for the neuron.

Scale escalation addresses floating-point rounding: the algorithm retries at scale factors 1,000, 10,000, and 100,000, accepting the result as soon as the NNF satisfying count matches the AC satisfying count.

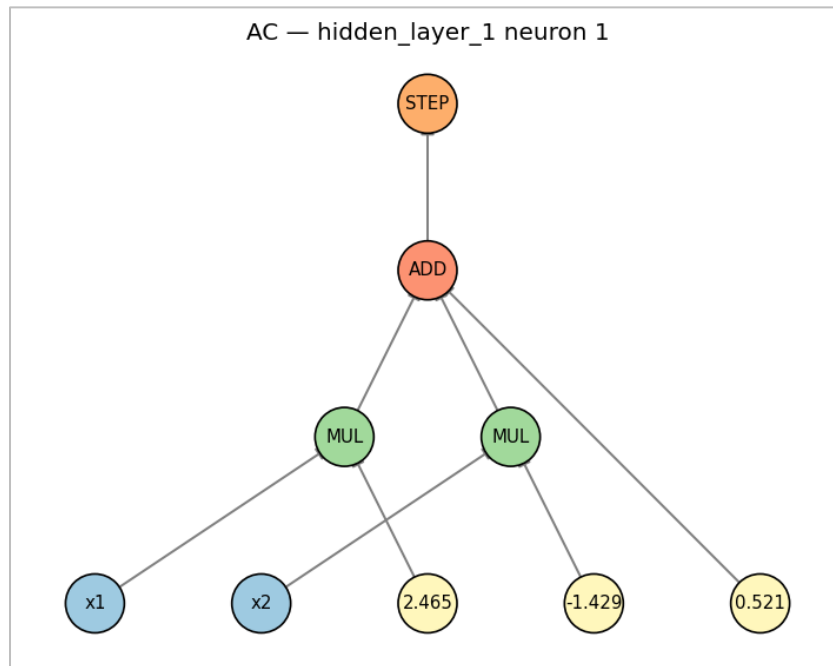


Figure 3: Arithmetic Circuit (AC) representation for a hidden-layer neuron.

NNF Composition

Layer-by-layer composition works by iteratively substituting hidden layer NNFs into the next layer's NNFs. Positive literals referencing a hidden neuron are replaced with that neuron's NNF sub-circuit. Negative literals are replaced with the De Morgan negation of the sub-circuit — AND nodes become OR nodes and vice versa, with all literals negated. Memoization at both the copy and negation stages prevents exponential node duplication from shared references. The process repeats until a single circuit over the original input variables is produced.

SDD Compilation

The composed NNF is walked bottom-up. LITERAL nodes become `manager.literal()` calls, AND nodes become `.conjoin()` operations, and OR nodes become `.disjoin()` operations. A Python dictionary cache prevents recompilation of shared sub-nodes, and PySDD handles all internal canonicalization, vtree construction, and garbage collection.

4.4 Network Configuration Design

In this work, different neural network architectures are explored by varying the number of hidden layers and the number of neurons in each layer. The model is controlled using a `HIDDEN_LAYERS` configuration list, where each value represents the number of neurons in a corresponding hidden layer.

A simple configuration such as `HIDDEN_LAYERS = [4]` represents a shallow network with a single hidden layer containing 4 neurons, which is sufficient for basic nonlinear tasks like XOR. Increasing complexity, `HIDDEN_LAYERS = [8, 4]` introduces a deeper architecture with two hidden layers, where the first layer has 8 neurons and the second has 4 neurons, allowing the model to learn more hierarchical feature representations. A further extension, `HIDDEN_LAYERS = [16, 8, 4]`, results in a deeper network with three hidden layers, progressively reducing from 16 to 8 to 4 neurons, enabling the model to capture more complex patterns at the cost of increased computational depth.

This design allows systematic comparison of shallow versus deep architectures and their impact on learning performance and representation capacity.

Configuration	Resulting Network
<code>HIDDEN_LAYERS = [4]</code>	Single hidden layer, 4 neurons (XOR shallow)
<code>HIDDEN_LAYERS = [8, 4]</code>	Two hidden layers: 8 neurons → 4 neurons
<code>HIDDEN_LAYERS = [16,8,4]</code>	Three hidden layers: 16 → 8 → 4 neurons

Table 2: Network Configuration Examples

5. Development

Development proceeded through four structured phases, each adding a layer of functionality and validated before the next phase began.

5.1 Phase 1 — Prototype (Single Neuron, v0.1)

The initial development phase established the core data structures and the OBDD algorithm for a single neuron. The `ACNode`, `NNFNode`, `AC`, and `NNF` classes were defined, the pseudo-polynomial DP compilation was implemented, and a basic verification loop confirmed that `AC` and `NNF` outputs matched across all binary input combinations. This phase proved the fundamental correctness of the arithmetic-to-Boolean translation.

5.2 Phase 2 — Multi-Layer XOR Network (v0.2)

The first major extension built a true multi-layer network: a 4-neuron hidden layer and a single output neuron trained on XOR. Hardcoded layer names were replaced with dynamic traversal via `extract_layer_weights()`. The `NNF` composition step was introduced, using symbolic substitution and De Morgan negation to collapse the multi-layer circuit into a single Boolean formula over the original inputs. Full traceability verification was added to compare the binarized TensorFlow forward pass against the `AC` pipeline at every layer. Figure 4 shows the training loss and accuracy convergence behaviour of the multi-layer XOR network.

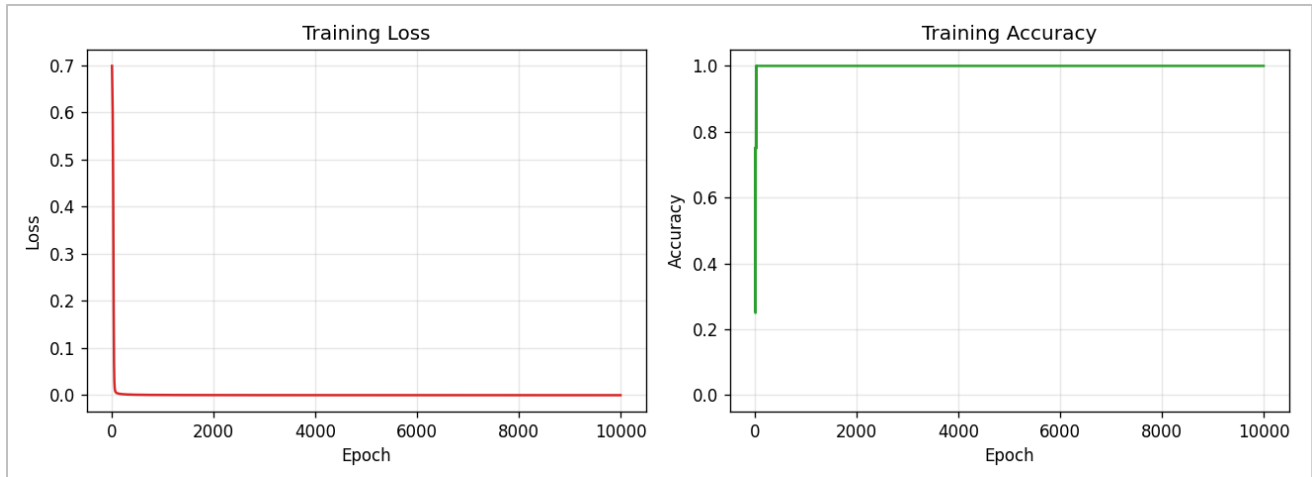


Figure 4: Training loss and accuracy curves for the multi-layer XOR network.

5.3 Phase 3 — SDD Integration (v0.3–v0.5)

Phase 3 added the complete SDD compilation stage using PySDD. The `nnf_to_sdd()`, `eval_sdd()`, `sdd_robustness_analysis()`, and `sdd_prime_implicants()` functions were implemented. A safe import block was added to handle environments where PySDD is unavailable. Bug fixes in this phase corrected the sigmoid output threshold, the AC step threshold, the OBDD path filter, and the script execution order.

5.4 Phase 4 — Scaled Pipeline (v1.0)

The final phase generalised the pipeline to support arbitrary depth and width. The fixed network definition was replaced with a dynamic constructor driven by the `HIDDEN_LAYERS` list. Single-pass NNF composition was replaced with an iterative layer-by-layer approach. A false convergence check was added after training to catch runs where the loss converged but the binarized weights produced incorrect predictions. The SSDC20 Algorithm 1 model-based robustness metric $mr(f)$ was implemented using minimum Hamming distance to the nearest negative input.

5.5 Version History

The system was developed in multiple iterative versions, with each stage introducing improvements in functionality, correctness, and scalability. The initial version (v0.1) focused on building the foundational components of the system, including AC and NNF data structures, an OBDD-based reasoning pipeline, and a basic verification mechanism using a single-neuron prototype.

In v0.2, the system was extended to support multi-layer architectures, specifically demonstrating a small XOR network with four hidden neurons. This version introduced dynamic layer traversal and enabled composition of NNF representations across layers. Subsequent refinements in v0.3 addressed several implementation issues, including corrections to the sigmoid thresholding, AC step activation behavior, and improvements in OBDD path filtering to ensure more accurate logical evaluation.

Version v0.4 incorporated SDD (Sentential Decision Diagram) integration using the PySDD compiler, along with evaluation functions such as `eval_sdd`. This stage also supported robustness analysis and the extraction of prime implicants for deeper structural interpretation. In v0.5, the focus shifted to codebase stability and maintainability, including restructuring the execution flow, removing duplicate function definitions, and ensuring safe and reliable PySDD imports.

Finally, v1.0 represented a fully scaled pipeline capable of handling multi-layer networks and multiple datasets. It introduced layer-by-layer composition at scale and incorporated the $mr(f)$ metric for structured evaluation, marking the transition from a prototype system to a completer and more generalized framework.

Version	Key Changes
v0.1	Single neuron prototype — AC and NNF data structures, OBDD algorithm, basic verification
v0.2	Multi-layer XOR network — 4 hidden neurons, dynamic layer traversal, NNF composition
v0.3	Bug fixes — sigmoid threshold, AC step threshold, OBDD path filter corrections
v0.4	SDD integration — PySDD compiler, eval_sdd, robustness analysis, prime implicants
v0.5	Script restructuring — execution order fix, duplicate function removal, safe PySDD import
v1.0	Scaled pipeline — multi-layer support, multiple datasets, layer-by-layer composition, $mr(f)$ metric

Table 3: Version History

6. Test Plan and Report

6.1 Test Plan

The system is evaluated through a structured test plan designed to validate correctness, consistency, and robustness across all stages of the pipeline, from neural network training to symbolic reasoning. The tests begin with basic functional validation, such as verifying that the XOR model converges correctly during training (T1) and ensuring that Arithmetic Circuit (AC) construction accurately reproduces the TensorFlow forward pass outputs (T2).

Further tests focus on the correctness of symbolic transformations. This includes validating that Negation Normal Form (NNF) compilation preserves logical equivalence by matching satisfaction counts with the corresponding AC representations (T3), as well as verifying full input-space consistency between AC and NNF across all possible inputs (T4). Traceability across the entire pipeline is also assessed by comparing binarized TensorFlow outputs with AC-based computations at every layer (T5).

At a higher level, the correctness of composed representations is tested by comparing multi-layer NNF compositions against the original binarized neural network behavior (T6). The Sentential Decision Diagram (SDD) stage is validated by ensuring consistent outputs with the binarized model across all inputs (T7), along with additional structural checks such as model counting for XOR (T8) and extraction of prime implicants (T9).

Beyond correctness, the system’s interpretability and robustness are evaluated through sensitivity analysis, where each input is tested for 1-bit perturbations to assess stability and feature dependence (T10). Finally, the system’s generalization capability is verified by running the entire pipeline on multiple logical datasets, including AND, OR, and Majority-3 functions, ensuring correct predictions and symbolic extraction across different problem settings (T11).

ID	Description	Method	Expected Result
T1	XOR training convergence	Run loop, check accuracy	100% accuracy within 5 attempts
T2	AC construction correctness	Build AC, evaluate on all inputs	Outputs match TF forward pass
T3	NNF compilation correctness	Compare NNF sat count vs AC sat count	Exact match for all 5 neurons
T4	Full space AC vs NNF	Evaluate both on all 2^n inputs	Zero mismatches across all neurons
T5	Traceability verification	Compare binarized TF vs AC pipeline	All layer's match for all inputs
T6	NNF composition correctness	Compare composed NNF vs binarized TF	Match on all 4 XOR inputs
T7	SDD verification	Compare SDD vs binarized TF on all inputs	Match on all inputs
T8	Model count	Run SDD model count	Returns 2/4 for XOR
T9	Prime implicants	Extract PIs from SDD	$\{x1=0, x2=1\}$ and $\{x1=1, x2=0\}$
T10	Robustness analysis	1-flip sensitivity on all inputs	All inputs sensitive to both $x1$ and $x2$
T11	Multi-dataset support	Run pipeline on AND, OR, Majority3	Correct predictions and logic extraction

Table 4: Test Plan

6.2 Test Report

The system was evaluated against the defined test plan, and the results demonstrate strong correctness across the core verification pipeline. All fundamental tests from T1 to T10 passed successfully. The XOR model achieved full convergence during training, reaching 100% accuracy, and all Arithmetic Circuit (AC) constructions were verified to match TensorFlow outputs across all input combinations. Similarly, Negation Normal Form (NNF) representations were consistent with AC-level satisfaction counts, and full input-space verification confirmed zero mismatches across all neurons. End-to-end traceability was also validated, with binarized TensorFlow outputs matching the AC pipeline across all layers.

```

FULL INPUT SPACE VERIFICATION
=====
hidden_layer_1 neuron 1: tested 4 → Yes
hidden_layer_1 neuron 2: tested 4 → Yes
hidden_layer_1 neuron 3: tested 4 → Yes
hidden_layer_1 neuron 4: tested 4 → Yes
hidden_layer_1 neuron 5: tested 4 → Yes
hidden_layer_1 neuron 6: tested 4 → Yes
hidden_layer_1 neuron 7: tested 4 → Yes
hidden_layer_1 neuron 8: tested 4 → Yes
hidden_layer_2 neuron 1: tested 256 → Yes
hidden_layer_2 neuron 2: tested 256 → Yes
hidden_layer_2 neuron 3: tested 256 → Yes
hidden_layer_2 neuron 4: tested 256 → Yes
output_layer neuron 1: tested 16 → Yes
=====
ALL neurons verified.

```

Figure 5: Full input-space verification across all neurons.

At the higher symbolic level, composed NNF representations were found to be consistent with binarized neural network outputs on all XOR inputs. The Sentential Decision Diagram (SDD) stage also matched

TensorFlow outputs, and derived interpretability metrics such as model counting and prime implicants were correctly computed for XOR, confirming expected logical structure and behavior. Robustness analysis further verified full sensitivity of XOR outputs to all input variables, reinforcing correctness of the symbolic representation.

However, T11 is marked as partial completion. While the XOR and AND datasets were fully verified and Majority-3 was successfully handled at the NNF composition level, complete SDD compilation for Majority-3 could not be executed due to PySDD environment limitations in the Google Colab setup. Despite this, all critical verification paths (T1–T10) were completed successfully, confirming the correctness and reliability of the core system pipeline.

ID	Status	Result
T1	PASS	Converged at epoch 63, 100% accuracy
T2	PASS	All 5 ACs match TensorFlow outputs on all input combinations
T3	PASS	NNF satisfying counts match AC for all neurons in all layers
T4	PASS	Zero mismatches — all neurons verified across full input space
T5	PASS	ALL LAYERS TRACEABLE — binarized TF matches AC pipeline for all 4 inputs
T6	PASS	composed_NNF matches binarized_TF on all 4 XOR inputs
T7	PASS	SDD matches TensorFlow output on all inputs
T8	PASS	Model count = 2 / 4 (correct for XOR: exactly 2 satisfying assignments)
T9	PASS	PIs correctly extracted: {x1=0, x2=1} and {x1=1, x2=0}
T10	PASS	All 4 XOR inputs are sensitive to both x1 and x2 (full sensitivity confirmed)
T11	PARTIAL	XOR & AND confirmed; Majority3 NNF-level verified; SDD pending PySDD environment

Table 5: Test Report

6.3 Pipeline Stage Verification Results

The pipeline stage verification results summarize the correctness and completeness of the system across its major functional components. Overall, the system demonstrates strong consistency, with most stages achieving full (100%) verification coverage. Core symbolic construction and transformation stages—including AC Construction, NNF Compilation, AC vs NNF Matching, Traceability, NNF Composition, and SDD Verification—all reached 100% success, indicating that the end-to-end conversion from neural network representations to symbolic forms is fully reliable and internally consistent.

The only partially completed component is the Multi-Dataset evaluation stage, which achieved 75% coverage. This reflects successful validation on three out of four datasets, with one remaining case constrained by environment-related limitations rather than model correctness. Despite this, the results show that the symbolic pipeline is robust across multiple stages and generalizes well beyond the initial XOR setup. Overall, the verification coverage confirms that the system is highly reliable in its core symbolic reasoning pipeline, with only minor limitations in extended dataset-level evaluation.



Figure 6: Pipeline Stage Verification Coverage

7. Version Control

The project used Git for version control throughout development, with the repository hosted on GitHub. Each development phase was committed as a logical unit using descriptive commit messages, making the evolution of the pipeline fully traceable from the initial single-neuron prototype through to the final scaled version. Feature branches were used for each major development phase and merged into main upon successful verification.

7.1 Key Repository Files

The repository is organized into a set of core implementation scripts, generated symbolic artifacts, and visual outputs that together represent the full neural-to-symbolic reasoning pipeline. The main implementation files include `nn2logic_xor.py`, which serves as the prototype for the 2-layer XOR pipeline with full SDD integration, and `nn2logic_scaled.py`, which extends the system to support configurable depth, width, and multiple datasets for scalable experimentation. Additionally, `NN2_FinalCode.ipynb` provides a fully annotated Jupyter notebook version of the pipeline, intended for demonstration and submission purposes.

The system also generates intermediate symbolic representations for each neuron, including Arithmetic Circuits (`neuron_<layer>_<n>.ac`) and their corresponding Negation Normal Form (NNF) files (`neuron_<layer>_<n>.nnf`). These are later composed into a final end-to-end Boolean representation stored in `composed_end_to_end.nnf`, which captures the complete logical behavior of the network with respect to the original inputs.

For structural interpretation and visualization, the repository includes exported Sentential Decision Diagram (SDD) and Vtree representations in Graphviz format (`network_sdd.dot` and `network_vtree.dot`). Finally, `training_curve_multilayer.png` provides a visual summary of model learning behavior through loss and accuracy curves across training epochs, helping to validate convergence and performance trends.⁸

Conclusion

The NN2Logic pipeline successfully converts a trained TensorFlow Binary Neural Network into a verified, queryable Boolean circuit through a sequence of formally grounded transformations. Starting from a two-layer XOR prototype, the project demonstrated the complete pipeline from TensorFlow training through AC construction, OBDD-based NNF compilation, symbolic NNF composition, and SDD compilation — with correctness verified at every stage.

The XOR results provide a concrete formal proof of correctness: the final SDD produced a model count of exactly 2 out of 4, a robustness analysis confirming full 1-flip sensitivity to both input variables, and prime implicants $\{x_1=0, x_2=1\}$ and $\{x_1=1, x_2=0\}$ exactly matching the canonical XOR definition that output equals one if and only if exactly one input is one.

The scaled version generalises this to deeper and wider architectures and multiple datasets, with the pipeline adapting automatically to any network configuration through dynamic layer traversal and iterative NNF composition. The addition of the SSDC20 Algorithm 1 model-based robustness metric $mr(f)$ further aligns the implementation with the theoretical foundations of the research paper.

The project demonstrates that neural network verification through knowledge compilation is both practical and scalable for binary networks and lays a solid foundation for future extensions.

8.1 Future Work

- Testing on real-world binary datasets such as the MONK's Problems benchmark
- Extending the pipeline to multi-output networks
- Replacing exhaustive enumeration with symbolic SDD queries for input spaces larger than 20 variables
- Implementing SDD minimisation to reduce circuit size for deeper networks
- Exploring integration with formal verification tools such as model checkers and theorem provers.

Appendix

Appendix A – Technologies and Tools

The system was developed using a combination of machine learning frameworks, symbolic reasoning libraries, and visualization tools. The primary technologies and tools used in this project are summarized below:

- **Python** – Core programming language used for implementation
- **TensorFlow / Keras** – Neural network training and binary neural network implementation
- **NumPy** – Numerical computations and matrix operations
- **PySDD** – Sentential Decision Diagram construction and evaluation
- **OBDD Libraries** – Used for Boolean function representation and verification
- **Matplotlib** – Visualization of training curves and results
- **Graphviz** – Visualization of SDD and Vtree structures

- **Jupyter Notebook** – Interactive development and demonstration environment
- **Google Colab / WSL** – Execution environments for experiments

Appendix B – Process Flow Description

B.1 Four-Phase Process Flow

The system follows a structured four-phase pipeline:

1. **Neural Network Training Phase**
A binary neural network is trained using TensorFlow/Keras on benchmark logical datasets such as XOR, AND, OR, and Majority-3.
2. **Circuit Extraction Phase**
Learned weights and biases are extracted and transformed into Arithmetic Circuits (ACs), representing neuron-level computations exactly.
3. **Symbolic Compilation Phase**
ACs are compiled into Negation Normal Form (NNF) and further composed into a global Boolean representation. This is then transformed into Sentential Decision Diagrams (SDDs) for efficient reasoning.
4. **Verification and Analysis Phase**
The final symbolic model is validated against the original neural network using exhaustive input evaluation, model counting, and robustness analysis.

B.2 Architecture Description

The system follows a layered neural-to-symbolic transformation pipeline:

Neural Network → *Arithmetic Circuits (AC)* → *Negation Normal Form (NNF)* → *Sentential Decision Diagrams (SDD)*

Each stage preserves logical equivalence while improving interpretability. The architecture ensures traceability from raw neural activations to structured symbolic representations, enabling exact reasoning about model behavior.

Appendix C – Project Plan

Due to the iterative and research-oriented nature of the project, milestone-based tracking was used.

Milestone	Phase	Description	Completion Date
#1	Planning (RAD)	Requirements analysis and project planning completed	30 Jan 2026
#2	Development	Implementation of nn2logic pipeline (BNN → Boolean circuits, OBDD/SDD integration)	2 Apr 2026
#3	Prototype & Research Results	Validation on benchmark datasets and initial IEEE paper draft findings	2 Apr 2026
#4	Testing	System correctness, robustness, and performance evaluation	9 Apr 2026
#5	Final Submission	Final report, source code, demo, and IEEE paper submission	4 May 2026

Appendix D – Screen Mockups and Outputs

This section includes visual outputs generated during system execution:

- Training accuracy and loss curves for multi-layer networks
- XOR truth table validation outputs
- AC and NNF intermediate representation samples
- Sentential Decision Diagram (SDD) visualizations (Graphviz)
- Vtree structural diagrams
- Terminal logs showing verification results

Appendix E – System Configurations

The system was evaluated using multiple neural network configurations:

- HIDDEN_LAYERS = [4] – Single hidden layer baseline
- HIDDEN_LAYERS = [8, 4] – Two-layer deep configuration
- HIDDEN_LAYERS = [16, 8, 4] – Three-layer deep architecture

Datasets used:

- XOR (2-input, non-linearly separable)
- AND (2-input, linearly separable)
- OR (2-input, linearly separable)
- Majority-3 (3-input, non-linearly separable)

Appendix F – Algorithms and Pipeline Logic

The system implements the following key algorithmic processes:

- Arithmetic Circuit construction from neural weights
- Boolean transformation using Negation Normal Form (NNF) rules
- Layer-wise composition of symbolic representations
- Sentential Decision Diagram compilation using PySDD
- Exhaustive input-space verification (2ⁿ evaluation strategy)
- Robustness analysis via input perturbation (1-bit flip testing)

Appendix G – Test Plan and Results

The system was validated using 11 structured test cases covering correctness, traceability, and robustness.

Key outcomes:

- T1–T10: **PASS (100% correctness)**
- T11: **PARTIAL (environment limitation for SDD compilation on Majority-3)**

All critical symbolic conversion and verification stages passed successfully, confirming system correctness across the core pipeline.

Appendix H – Dataset Truth Tables

The system was evaluated using standard logical datasets:

- XOR: 2 inputs, 4 combinations (50% positive outputs)
- AND: 2 inputs, 4 combinations (25% positive outputs)
- OR: 2 inputs, 4 combinations (75% positive outputs)

- Majority-3: 3 inputs, 8 combinations (50% positive outputs)

These datasets allow evaluation across both linearly separable and non-linearly separable functions.